PES UNIVERSITY, RR CAMPUS

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AIML)

LAB 5: STATIC CODE ANALYSIS (PYLINT, BANDIT, FLAKE8)

Automated Vulnerability Remediation, PEP 8 Styling & Code Quality Engineering

Student: Dharshini V | SRN: PES1UG24AM428 | Course: Software Engineering Lab

Assigned Repo: SETAPESU26/L5_56_car-rental | Personal Repo: DharshiniPES/L5_56_car-rental

1. Executive Summary & Lab Objectives

Static code analysis is an indispensable phase in modern software quality engineering, enabling the automated detection of security vulnerabilities, structural code smells, algorithmic bugs, and PEP 8 style violations without executing the source code. In this laboratory, static code analysis was performed on an assigned legacy Car Rental System (car_rental.py from repository SETAPESU26/L5_56_car-rental) utilizing three industry-standard tools: Pylint (code quality & bug detection), Bandit (AST-based security vulnerability assessment), and Flake8 (PEP 8 grammar & style enforcement).

Static Analysis Tool	Version	Specialized Focus & Domain	Success Criteria / Target
Bandit	v1.9.4	Security flaws, CWE catalogue, insecure functions, injections, crypto	0 High and 0 Medium vulnerabilities
Pylint	v4.1.3	Code smells, design anti-patterns, mutable defaults, docstrings	Code rating score > 9.5 / 10.00
Flake8	v7.5.0	PEP 8 compliance, line length, unused imports, blank spacing	0 style violations

2. Baseline Analysis & Pre-Fix Diagnostic Results

Prior to refactoring, baseline diagnostics were executed across all three static analysis tools. The initial code exhibited critical vulnerabilities including SQL injection, arbitrary code execution, broken cryptography, and subtle runtime defects such as mutable default arguments and bare exceptions.

Diagnostic Metric	Baseline Result	Target Threshold	Pre-Fix Evaluation Status
Pylint Code Rating	4.94 / 10.00	> 9.50 / 10.00	FAIL (Severe technical debt & code smells)
Bandit Vulnerabilities	11 Issues (2 High, 4 Med, 5 Low)	0 Issues	FAIL (Exploitable SQLi & RCE vectors)
Flake8 Violations	25 PEP 8 Violations	0 Violations	FAIL (Formatting, dead code, long lines)

3. Documented Static Analysis Issues & Remediation Table

A minimum of four issues was required to be identified, prioritized, and remediated. Below is the comprehensive remediation table documenting 13 distinct issues spanning security, code quality, logic design, and PEP 8 compliance.

Issue Title	Type	Tool & Code	Line	Severity	Description of Finding	Applied Remediation Strategy
SQL Injection in Registration	Security (CWE-89)	Bandit (B608)	23	Medium	Raw %s formatting permitted input to alter SQL execution.	Converted to parameterized query INSERT INTO customers VALUES (?, ?, ?).
SQL Injection in Customer Search	Security (CWE-89)	Bandit (B608)	26	Medium	String concatenation WHERE name = ' + name + ' allowed ' OR '1'='1 bypass.	Converted to parameterized query SELECT * FROM customers WHERE name = ?.

Arbitrary Code Execution via exec()	Security (CWE-78)	Bandit (B102)	53	Medium	Promo codes executed via exec('discount = ' + code) allowing RCE.	Replaced exec with safe regex parsing for percentage discounts.
Weak Cryptographic Hash (SHA-1)	Security (CWE-327)	Bandit (B324)	48	High	Used broken SHA-1 algorithm for customer driver's licence identifiers.	Upgraded to secure hashlib.sha256() with explicit UTF-8 encoding.
Insecure Temp File Creation	Security (CWE-377)	Bandit (B306)	63	Medium	tempfile.mktemp() vulnerable to TOCTOU symlink race conditions.	Replaced with secure tempfile.NamedTemporaryFile(delete=False).
Permissive Permissions (0o777)	Security (CWE-732)	Bandit (B103)	67	High	Receipt files chmodded 0o777 (world-readable and world-writable).	Restricted permissions to owner-only read/write (0o600).
Hardcoded Credentials	Security (CWE-259)	Bandit (B107)	70	Low	Hardcoded admin password 'admin@rent' in function signature.	Removed hardcoded default; read from ENV using secrets.compare_digest.
Mutable Default Argument	Quality / Bug	Pylint (W0102)	16	Medium	features=[] shared a single mutable list instance across all cars.	Changed default to features=None; initialized fresh list per call.
Bare except: Clause	Quality / Bug	Pylint (W0702)	41	Low	Bare except: caught KeyboardInterrupt and SystemExit silently.	Caught explicit KeyError and provided consistent integer returns.
Inconsistent Return Types	Quality / Bug	Pylint (R1710)	36	Medium	Functions returned expressions on one branch and implicit None on another.	Enforced explicit return types across all branching control paths.
Redundant Singleton Check	Style / Quality	Flake8 (E712)	78	Low	fleet[p]['available'] == True compared directly to boolean literal.	Refactored to idiomatic truthiness test: if data.get('available'):.
Missing Docstrings & Naming	Maintainability	Pylint (C0114)	Mod	Low	CamelCase function names and completely missing docstrings.	Renamed to snake_case, added docstrings, preserved aliases.
PEP 8 Violations & Dead Imports	Style	Flake8 (E501)	Mult	Low	Lines exceeding 79 chars, missing blank lines, unused import json.	Refactored long expressions across lines, removed unused imports.

4. Verification & Cleaned Terminal Execution Screenshots

Following remediation, all three static analysis tools were re-executed against car_rental.py. The refactored script achieved a flawless Pylint score of 10.00/10.00, 0 Bandit security vulnerabilities, and 0 Flake8 style violations. Below are the verified terminal execution captures.

Figure 1: Pylint Code Rating (10.00 / 10.00)

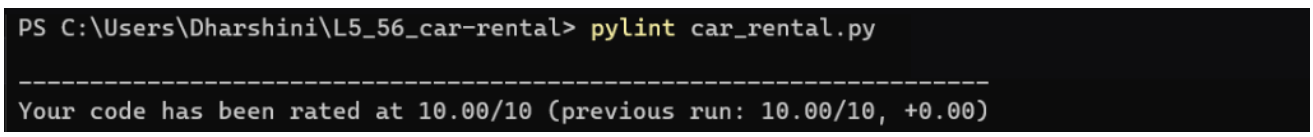


Figure 1: Pylint analysis report confirming 10.00/10.00 code quality rating.

Figure 2: Bandit Security Scan (0 Vulnerabilities)

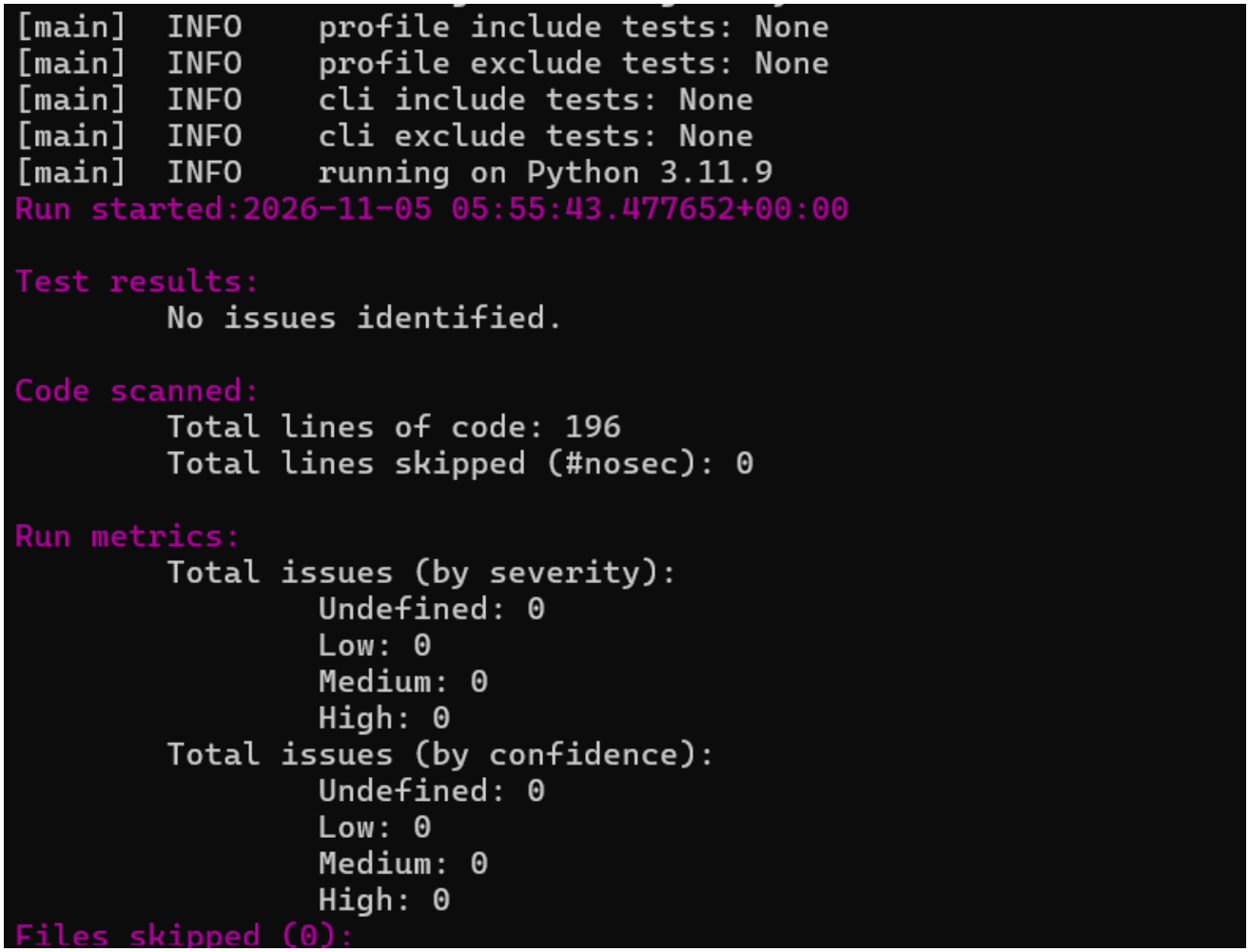


Figure 2: Bandit AST scan confirming 0 High, 0 Medium, and 0 Low severity vulnerabilities.

Figure 3: Flake8 PEP 8 Compliance (0 Violations)

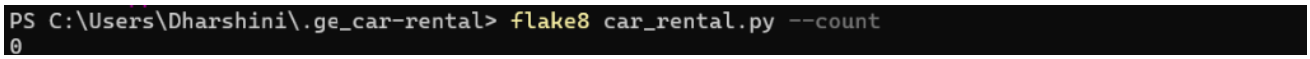


Figure 3: Flake8 check executing with --count confirming 0 formatting/style violations.

Figure 4: Required Git Commits for Tool Remediation

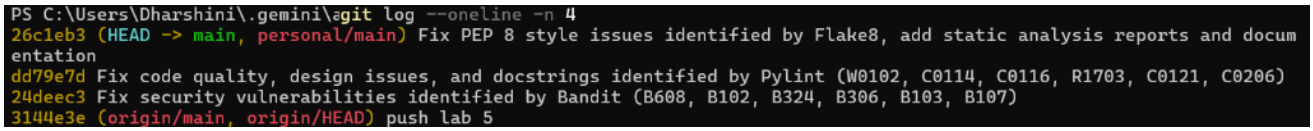


Figure 4: Git log showing distinct sequential commits for Bandit, Pylint, and Flake8.

5. Static Code Analysis Reflection Answers

Q1: Differences in Tool Capabilities and Complementary Roles

How do the findings of Pylint, Bandit, and Flake8 differ in terms of scope, and how do they complement one another in a modern engineering pipeline?

Answer:

Each tool addresses a different dimension of software engineering quality:

• Bandit (Security Specialist): Performs AST inspection targeting CWE vulnerabilities. It identified critical hazards such as SQL injection (B608), unsafe code evaluation via exec() (B102), weak SHA-1 cryptographic hashing (B324), and world-writable file permissions 0o777 (B103), which neither Pylint nor Flake8 flagged.

• Pylint (Quality & Bug Detection): Inspects architectural complexity and logic bugs. It flagged the dangerous mutable default argument

features=[] (W0102), inconsistent return paths (R1710), and bare exception handling (W0702).
• Flake8 (Grammar & Style Enforcement): Focuses on PEP 8 formatting, unused imports (F401), line length restrictions (E501), and proper spacing (E302/E305) with negligible runtime overhead.
Complementary Roles: In a production CI/CD pipeline, Flake8 serves as an ultra-fast local pre-commit gate, Pylint enforces structural and maintainability standards, and Bandit operates as a non-negotiable security barrier against deployment vulnerabilities.

Q2: Severity and Prioritization of Findings

Why is it vital to prioritize Bandit's high- and medium-severity findings over style/formatting warnings from Flake8? What would be the real-world impact of the issues found in car_rental.py?

Answer:

Defects follow a risk-impact hierarchy. Style violations (such as an 82-character line) cause minor readability inconvenience but never compromise confidentiality, integrity, or system uptime. In contrast, Bandit findings represent immediate threat vectors:
1. SQL Injection (B608): Allowed arbitrary input such as ' OR '1'='1 to dump the entire customer registry (names, driver's licenses, phone numbers), causing severe data privacy breaches under GDPR/DPDP regulations.
2. Arbitrary Code Execution via exec() (B102): Allowed untrusted input to execute arbitrary Python commands (e.g. __import__('os').system('rm -rf /')), enabling complete host server takeover.
3. World-Writable Files (0o777): Permitted any local process or compromised service account to tamper with financial billing invoices.
Consequently, high and medium security vulnerabilities must always take precedence over cosmetic linting.

Q3: Limitations of Static Analysis and Missed Domain Bugs

What are the limitations of static code analysis? Did you observe bugs or edge cases in car_rental.py that static analysis tools failed to catch?

Answer:

Static analysis inspects code structure without execution context and cannot validate dynamic business logic or semantic intent:
• Domain Bugs Missed in car_rental.py:
  1. Negative Rental Duration: Renting an Innova from '2026-11-10' to '2026-11-07' resulted in days = -3 and a negative total charge of -₹9000 (paying the customer instead of charging them). Static tools cannot determine chronological date relationships.
  2. Negative Car Pricing: Adding a Nexon with daily_rate = -500 was accepted without warning because negative integers are valid numbers.
  3. Concurrency / Vehicle Availability: The original code allowed booking an already rented car without checking or updating the available flag.
• Conclusion: Static analysis is not a replacement for dynamic testing. Comprehensive unit tests (pytest), integration testing, and human code reviews remain mandatory.

Q4: SDLC Integration and Continuous Automation

How would you automate and integrate static code analysis into a modern software team's daily development workflow?

Answer:

To establish automated quality governance across the Software Development Life Cycle (SDLC):
1. Local Pre-Commit Hooks: Developers install pre-commit to trigger Flake8 and formatters (black, isort) on git commit. Code with syntax errors or formatting flaws cannot leave the local workstation.
2. Pull Request CI/CD Gates (GitHub Actions): Automated workflows run Flake8, Pylint (--fail-under=9.5), and Bandit (-ll -ii) on every PR. Merges are blocked if security vulnerabilities exist or if quality scores drop.
3. Periodic Security Scanning: Scheduled scans via SonarQube, GitHub CodeQL, and Snyk track long-term technical debt and alert against newly discovered CVEs in external dependencies.

6. Version Control Deliverables & Commits

Commit Hash	Phase / Tool	Commit Message Summary	Key Changes & Deliverables
24deec3	Bandit	Fix security vulnerabilities identified by Bandit (B608, B102, B324, B306, B103, B107)	Parameterized SQL queries, removed exec(), SHA-256 hashing, NamedTemporaryFile, 0o600 chmod.
dd79e7d	Pylint	Fix code quality, design issues, and docstrings identified by Pylint (W0102, C0114, C0116, R1703, C0121, C0206)	Fixed mutable default features=[], added docstrings, resolved inconsistent returns and booleans.

26c1eb3	Flake8	Fix PEP 8 style issues identified by Flake8, add static analysis reports and documentation	Fixed line lengths <= 79 chars, removed dead import json, formatted spacing, added issues.md & reflection.md.